

1 Partición Horizontal

Consiste en dividir las filas (**registros**) de una tabla en varias subtablas, manteniendo todas las columnas.

◆ Ejemplo:

Imagina una base de datos con información de clientes. En lugar de almacenar todos los clientes en una sola tabla, puedes dividirlos según su ubicación:

- Tabla 1: Clientes de América
- Tabla 2: Clientes de Europa
- Tabla 3: Clientes de Asia

◆ Ventajas:

- ✓ Mejora el rendimiento en bases de datos distribuidas, ya que cada servidor maneja menos datos.
- ✓ Es útil cuando se accede a los datos en función de una característica específica (como la ubicación).
- ✓ Reduce la carga de consultas en bases de datos grandes.

◆ Desventajas:

- ⚠ Puede generar problemas de equilibrio si algunas particiones tienen muchos más datos que otras.
- ⚠ Si se necesita consultar toda la información, la consulta puede volverse más costosa.



2 Partición Vertical

Consiste en dividir las **columnas** de una tabla en diferentes subtablas, manteniendo las mismas filas en cada una.

◆ Ejemplo:

Si una tabla de clientes tiene muchas columnas (nombre, dirección, historial de compras, métodos de pago, etc.), podrías separarlas en:

- Tabla 1: Datos básicos del cliente (ID, nombre, dirección)
- Tabla 2: Historial de compras (ID, productos comprados, fechas)
- Tabla 3: Métodos de pago (ID, tarjetas registradas, cuentas bancarias)

◆ Ventajas:

✓ Reduce la cantidad de datos leídos en cada consulta si solo se necesita un subconjunto de información.

✓ Puede mejorar la eficiencia cuando diferentes tipos de datos se acceden con distintas frecuencias.

✓ Es útil para seguridad y cumplimiento, permitiendo restringir el acceso a ciertas tablas.

◆ Desventajas:

⚠ Puede aumentar la complejidad en las consultas, ya que requieren **joins** para recomponer la información completa.

⚠ Puede generar inconsistencias si no se diseñan  correctamente las relaciones entre las tablas.

Vertical

Horizontal

Partición Round Robin 🏓⚡

La **partición Round Robin** es una estrategia de **partición horizontal** en la que los datos se distribuyen de manera equitativa entre varias particiones o nodos siguiendo un orden cíclico.

◆ ¿Cómo funciona?

Imagina que tienes una base de datos grande y quieres dividirla en **N particiones**. En lugar de basarte en un criterio específico (como la ubicación o una clave de partición), asignas cada nuevo registro a una de las particiones de manera secuencial y cíclica.

📌 Ejemplo con 3 particiones:

Si ingresan los siguientes registros:

- 1 Registro A → Partición 1
- 2 Registro B → Partición 2
- 3 Registro C → Partición 3
- 4 Registro D → Partición 1 (se reinicia el ciclo)
- 5 Registro E → Partición 2
- 6 Registro F → Partición 3

Este proceso continúa **en ciclo**, asegurando que las particiones tengan un tamaño equilibrado.

